

EduGest

Documentation des Endpoints API

Plateforme de gestion d'école primaire et maternelle

Générée le 23/04/2026 · Version 1.0

123
Endpoints

15
Modules

33
Tables 3D

21
Critiques ★

Méthodes HTTP

GET	Lecture
POST	Création
PUT	Modification complète
PATCH	Modification partielle
DELETE	Suppression

★ Critique = endpoint dont la logique côté backend est spécifique au projet (jointures complexes, règles métier, notifications...)

Sommaire

Authentification	Admin + Personne	3 eps
Années Académiques	AnneeAcademique	3 eps
Élèves	Eleve	7 eps
Parents	Parents + Personne	5 eps
Classes, Cycles & Salles	Classe + Cycle + Salle	8 eps
Cours / Matières	Cours + Enseignant	9 eps
Emploi du Temps	EmploiDuTemps + JourSemaine	7 eps
Évaluations & Notes	Evaluation + Epreuve + NatureEpreuve	13 eps
Messagerie	Messages	10 eps
Paielements & Scolarité	Paielement + Scolarite + Tranches + Mode	17 eps
Salaires Enseignants	FicheEnseignant	7 eps
Rapports de Discipline & Absences	Rapport + Justificatifs + Discipline	7 eps
Bibliothèque	Livres + Specialite	7 eps

Statistiques & Rapports Admin	Aggregation multi-tables	2 eps
Données de Référence	VilleNaissance + Quartier + Residents + Titulaire + Fréquentation	18 eps

AU
TH

Authentification

Table(s) BD : Admin + Personne

3 endpoints

POST

/api/auth/login

Connexion

Corps : username, password

Réponse : { token, user: { idPers|ID, nom, prenom, role, typePersonne|typeAdmin } }

Note : Vérifie d'abord table Admin, puis Personne. Retourne le rôle selon typeAdmin ou typePersonne.

★ Critique

GET

/api/auth/me

Profil de l'utilisateur connecté

Corps : JWT Header

Réponse : { user }

Note : Requier JWT valide.

POST

/api/auth/change-password

Changer mot de passe

Corps : currentPassword, newPassword

Réponse : { message }

Note : Met à jour password dans Admin ou Personne selon le type.

AC
A

Années Académiques

Table(s) BD : AnneeAcademique

3 endpoints

GET

/api/annees-academiques

Liste de toutes les années

Réponse : { annees: [...] }

Note : Retourner idAnnee, libelle, periode.

GET

/api/annees-academiques/active

Année académique en cours

Réponse : { idAnnee, libelle, periode }

Note : Retourner la plus récente ou celle marquée active.

POST

/api/annees-academiques

Créer une année académique

Corps : libelle, periode

Réponse : { idAnnee, libelle }

Note : idAdmin extrait du JWT.

GET

/api/eleves**Liste des élèves**

Corps : ?search, ?page, ?limit

Réponse : { eleves: [...] }

Note : Champs: matricule, nom, prenom, sexe, actif, classe.

GET

/api/eleves/:matricule**Un élève par matricule**

Réponse : { eleve }

Note : Toutes les infos de l'élève + classe + cycle.

GET

/api/eleves/par-cours**Élèves d'un cours (pour saisie notes)**

Corps : ?idCours

Réponse : { eleves: [...] }

Note : JOIN Enseignant → Cours → Frequent → Eleve.

★ Critique

POST

/api/eleves**Inscrire un élève**

Corps : nom, prenom, dateNaissance, lieuNaissance, sexe, idVilleNaissance

Réponse : { matricule, message }

Note : idAdmin extrait du JWT. actif = 0 par défaut.

PUT

/api/eleves/:matricule**Modifier un élève**

Corps : champs à modifier

Réponse : { message }

DELE
TE**/api/eleves/:matricule****Supprimer un élève**

Réponse : { message }

Note : Vérifier qu'aucun paiement actif n'est lié.

PATC
H**/api/eleves/:matricule/actif****Activer / désactiver un élève**

Corps : actif: 0|1

Réponse : { message }

PA
RE
NT

Parents

Table(s) BD : Parents + Personne

5 endpoints

GET

/api/parents

Liste de tous les parents

Corps : ?search, ?page

Réponse : { parents: [...] }

Note : JOIN Personne. Inclure nom, prenom, mobile, enfants.

GET

/api/parents/mes-enfants

Enfants du parent connecté

Corps : JWT Header

Réponse : { enfants: [{ matricule, nom, prenom, classe, actif }] }

Note : idPers extrait du JWT → Parents → Eleve + Frequenté.

★ Critique

GET

/api/parents/:idParent/enfants

Enfants d'un parent précis

Réponse : { enfants: [...] }

POST

/api/parents

Créer un lien parent-élève

Corps : idPers, matricule

Réponse : { idParent }

Note : Vérifie unicité (idPers, matricule).

DELETE

/api/parents/:idParent

Supprimer un parent

Réponse : { message }

CL
AS
SE

Classes, Cycles & Salles

Table(s) BD : Classe + Cycle + Salle

8 endpoints

GET

/api/classes

Liste des classes

Corps : ?idCycle

Réponse : { classes: [{ idClasse, libelle, libelleCycle, nbEleves, nbCours }] }

Note : JOIN Cycle. Compter élèves via Frequenté.

POST

/api/classes

Créer une classe

Corps : libelle, idCycle

Réponse : { idClasse }

Note : idAdmin extrait du JWT.

PUT

/api/classes/:idClasse

Modifier une classe

Corps : libelle, idCycle

Réponse : { message }

DELETE

/api/classes/:idClasse

Supprimer une classe

Réponse : { message }

GET	/api/cycles Liste des cycles Réponse : { cycles: [{ idCycle, libelle, description }] }
POST	/api/cycles Créer un cycle Corps : libelle, description Réponse : { idCycle }
GET	/api/salles Liste des salles Corps : ?idClasse Réponse : { salles: [...] } <i>Note : Champs: idSalle, libelle, position, surface, actif.</i>
POST	/api/salles Créer une salle Corps : libelle, position, surface, idClasse Réponse : { idSalle }

CO UR S	Cours / Matières	9 endpoints
	Table(s) BD : Cours + Enseignant	
GET	/api/cours Liste des cours Corps : ?idClasse Réponse : { cours: [{ idCours, libelle, coefficient, actif, idClasse }] }	
GET	/api/enseignant/mes-cours Cours de l'enseignant connecté Corps : JWT Header Réponse : { cours: [...] } <i>Note : idPers → Enseignant → Cours.</i>	★ Critique
POST	/api/cours Créer un cours Corps : libelle, coefficient, description, idClasse Réponse : { idCours }	
PUT	/api/cours/:idCours Modifier un cours Corps : libelle, coefficient Réponse : { message }	
DELETE	/api/cours/:idCours Supprimer un cours Réponse : { message }	
GET	/api/enseignants Liste des enseignants Corps : ?search Réponse : { enseignants: [{ idEnseignant, nom, prenom, libelleCours, Actif }] } <i>Note : JOIN Personne + Cours.</i>	

POST	/api/enseignants Créer un enseignant Corps : nom, prenom, mobile, username, password, idCours Réponse : { idEnseignant } <i>Note : Crée d'abord dans Personne (typePersonne=1), puis Enseignant.</i>
PUT	/api/enseignants/:idEnseignant Modifier un enseignant Corps : champs Réponse : { message }
DELETE	/api/enseignants/:idEnseignant Supprimer un enseignant Réponse : { message }

EDT	Emploi du Temps	7 endpoints
Table(s) BD : EmploiDuTemps + JourSemaine		
GET	/api/emploi-du-temps EDT d'une classe Corps : ?idClasse, ?jour Réponse : { creneaux: [{ idTemps, jour, heure, libelleCours, libelleSalle }] } <i>Note : JOIN Cours + Salle.</i>	★ Critique
GET	/api/emploi-du-temps/enseignant EDT d'un enseignant Corps : ?idPers Réponse : { creneaux: [...] } <i>Note : Via Enseignant → Cours → EmploiDuTemps.</i>	★ Critique
GET	/api/emploi-du-temps/eleve EDT d'un élève Corps : ?matricule Réponse : { creneaux: [...] } <i>Note : Via Frequenté → Salle → Classe → EmploiDuTemps.</i>	
POST	/api/emploi-du-temps Créer un créneau Corps : jour, heure, idClasse, idCours Réponse : { idTemps }	
PUT	/api/emploi-du-temps/:idTemps Modifier un créneau Corps : jour, heure, idClasse, idCours Réponse : { message }	
DELETE	/api/emploi-du-temps/:idTemps Supprimer un créneau Réponse : { message }	
GET	/api/jours-semaine Liste des jours de la semaine Réponse : { jours: ['Lundi', ...] } <i>Note : Table JourSemaine.</i>	

NO TE	Évaluations & Notes	13 endpoints
Table(s) BD : Evaluation + Epreuve + NatureEpreuve		
GET	/api/evaluations Notes d'un élève Corps : ?matricule, ?idSession, ?idAca Réponse : { evaluations: [{ note, coefficient, appreciation, libelleCours, libelleNature }] } Note : JOIN Cours + NatureEpreuve + Epreuve.	★ Critique
GET	/api/evaluations/classe Notes d'une classe / cours Corps : ?idCours, ?idSession Réponse : { evaluations: [...] } Note : Pour la saisie en masse par l'enseignant.	★ Critique
POST	/api/evaluations Saisir une note Corps : note, appreciation, matricule, idEpreuve, idCours, idSession Réponse : { idEval } Note : idPers extrait du JWT.	
PUT	/api/evaluations/:idEval Modifier une note Corps : note, appreciation Réponse : { message } Note : Vérifier que idPers JWT = idPers de l'évaluation.	
DELETE	/api/evaluations/:idEval Supprimer une note Réponse : { message }	
GET	/api/epreuves Liste des épreuves / exercices Corps : ?idPers Réponse : { epreuves: [{ idEpreuve, libelle, auteur, libelleNature, urlDoc }] } Note : JOIN NatureEpreuve.	
GET	/api/epreuves/classe Épreuves accessibles à un élève Corps : ?matricule Réponse : { epreuves: [...] } Note : Via Frequente → Classe → Enseignant → Epreuve.	
POST	/api/epreuves Publier une épreuve / exercice Corps : libelle, urlDoc, auteur, idNature Réponse : { idEpreuve } Note : idPers extrait du JWT.	
GET	/api/natures-epreuve Types d'épreuves Réponse : { natures: [{ idNature, libelle }] } Note : Valeurs: Controle Continu, Examen, Devoir Mercredi, Devoir Week End.	

GET	/api/sessions Sessions d'un trimestre Corps : ?idTrimestre Réponse : { sessions: [...] }
GET	/api/sessions/actives Sessions actives Réponse : { sessions: [...] }
POST	/api/sessions Créer une session Corps : libelle, description, idTrimestre Réponse : { idSession }
GET	/api/trimestres Trimestres d'une année Corps : ?idAca Réponse : { trimestres: [...] }

MS G	Messagerie	10 endpoints
GET	/api/messages Messages d'un parent Corps : ?idParent Réponse : { messages: [{ idMessages, objet, information, type_message, valider, created_at }] } <i>Note : type_message: 0=individuel, 1=tous parents, 2=paiement.</i>	★ Critique
GET	/api/messages/all Tous les messages (admin) Corps : ?valider, ?limit Réponse : { messages: [...] } <i>Note : Réservé aux admins.</i>	
GET	/api/messages/recent Derniers messages (dashboard) Corps : ?limit Réponse : { messages: [...] }	★ Critique
GET	/api/messages/recus Messages reçus (enseignant) Corps : ?idPers Réponse : { messages: [...] }	★ Critique
GET	/api/messages/unread-count Compteur messages non lus Corps : ?idParent Réponse : { count: N }	
POST	/api/messages Envoyer un message Corps : objet, information, type_message, idParent, AnneeAcade Réponse : { idMessages } <i>Note : idExp_Pers extrait du JWT. valider=0 par défaut.</i>	

POST	/api/messages/reply Répondre à un message Corps : idMessageOriginal, information, idParent Réponse : { idMessages } <i>Note : Crée un nouveau message lié à l'original.</i>	★ Critique
POST	/api/messages/masse Message à tous les parents Corps : objet, information, type_message: 1 2 Réponse : { count } <i>Note : Insère N lignes (une par parent). Admin seulement.</i>	
PATCH	/api/messages/:id/valider Valider un message (admin) Réponse : { message } <i>Note : Met valider=1.</i>	
PATCH	/api/messages/:id/lire Marquer comme lu Réponse : { message }	

PAIE	Paieements & Scolaarité Table(s) BD : Paiement + Scolaarity + Tranches + Mode	17 endpoints
GET	/api/paiements Liste des paiements Corps : ?matricule, ?statut, ?idAca Réponse : { paiements: [{ idPaie, montant, valide, datePaie, libelleMode, nomEleve }] } <i>Note : JOIN Mode + Eleve.</i>	
GET	/api/paiements/mon-compte Paieements du parent connecté Corps : JWT Header Réponse : { paiements: [...] } <i>Note : idPers → Parents → matricule → Paiement.</i>	★ Critique
GET	/api/paiements/recent Derniers paiements (dashboard) Corps : ?limit Réponse : { paiements: [...] }	
GET	/api/paiements/:idPaie Un paiement précis Réponse : { paiement }	
POST	/api/paiements Enregistrer un paiement (admin) Corps : matricule, idAca, montant, idMode, operation_ID, datePaie Réponse : { idPaie } <i>Note : Enregistrement direct, valide=1 automatiquement.</i>	

POST	/api/paiements/initier Initier une demande (parent) Corps : matricule, idMode, idAca Réponse : { idPaie, message } <i>Note : valide=0. Notifie l'admin. Parent ne peut pas valider lui-même.</i>	★ Critique
PATCH	/api/paiements/:idPaie/valider Valider un paiement (admin) Réponse : { message } <i>Note : Met valide=1. Admin seulement.</i>	
GET	/api/scolarite Frais de scolarité par cycle Corps : ?idCycle Réponse : { scolarite: { idScolarite, inscription, pension, nbreTranche } }	★ Critique
POST	/api/scolarite Définir les frais Corps : inscription, pension, nbreTranche, description, idCycle Réponse : { idScolarite } <i>Note : idFondateur extrait du JWT.</i>	
PUT	/api/scolarite/:idScolarite Modifier les frais Corps : inscription, pension, nbreTranche Réponse : { message }	
GET	/api/tranches Tranches d'une scolarité Corps : ?idScolarite Réponse : { tranches: [{ idTranche, libelle, montant, delai_mois, delai_jour }] }	★ Critique
POST	/api/tranches Créer une tranche Corps : libelle, montant, delai_mois, delai_jour, idScolarite Réponse : { idTranche }	
PUT	/api/tranches/:idTranche Modifier une tranche Corps : libelle, montant, delai_mois, delai_jour Réponse : { message }	
DELETE	/api/tranches/:idTranche Supprimer une tranche Réponse : { message }	
GET	/api/modes-paiement Modes de paiement actifs Corps : ?actif Réponse : { modes: [{ idMode, libelle, information, actif }] }	
POST	/api/modes-paiement Créer un mode Corps : libelle, information Réponse : { idMode } <i>Note : idFondateur extrait du JWT.</i>	

PATC
H

/api/modes-paiement/:idMode
Activer/désactiver un mode
Corps : actif: 0|1
Réponse : { message }

SAL

Salaires Enseignants

7 endpoints

Table(s) BD : FicheEnseignant

GET

/api/enseignant/salaire
Historique salaires de l'enseignant connecté
Corps : JWT Header
Réponse : { salaires: [{ idRap, libelle, points, statut, event_date, commentaire }] }
Note : points = montant du salaire.

★ Critique

GET

/api/enseignant/salaire/statut
Statut du mois en cours
Corps : JWT Header
Réponse : { salaire: { statut, montant } }
Note : statut: disponible | en_attente | verse.

★ Critique

POST

/api/enseignant/salaire/decaissement
Demander le décaissement
Corps : idEnseignant
Réponse : { message }
Note : Notifie l'admin. L'admin doit valider (virement ou cash).

★ Critique

GET

/api/salaires
Tous les salaires (admin)
Corps : ?idEnseignant, ?idAca
Réponse : { salaires: [...] }
Note : JOIN FicheEnseignant + Personne.

POST

/api/fiches-enseignant
Créer une fiche salaire
Corps : idEnseignant, libelle, points, idAca, event_date, commentaire
Réponse : { idRap }
Note : idAdministratif extrait du JWT.

PATC
H

/api/salaires/:idFiche/valider
Valider le paiement du salaire
Corps : modeReglement: virement|cash
Réponse : { message }
Note : Admin valide après décaissement.

PUT

/api/fiches-enseignant/:idRap
Modifier une fiche
Corps : libelle, points, commentaire
Réponse : { message }

RA P	Rapports de Discipline & Absences	7 endpoints
Table(s) BD : Rapport + Justificatifs + Discipline		
GET	/api/rapports Liste des rapports Corps : ?matricule, ?idAca Réponse : { rapports: [{ idRap, libelle, points, event_date, commentaire, justifie, nomEleve }] } <i>Note : JOIN Eleve. Ajouter champ justifie (via Justificatifs).</i>	
GET	/api/rapports/cours Rapports d'un cours (enseignant) Corps : ?idCours Réponse : { rapports: [...] } <i>Note : Via Enseignant → Cours → Rapport.</i>	★ Critique
POST	/api/rapports Créer un rapport d'absence Corps : libelle, points, matricule, idAca, event_date, commentaire, idDiscipline Réponse : { idRap } <i>Note : idPers extrait du JWT.</i>	
PUT	/api/rapports/:idRap Modifier un rapport Corps : libelle, commentaire Réponse : { message }	
GET	/api/justificatifs Justificatifs d'un rapport Corps : ?idRapport Réponse : { justificatifs: [...] }	
POST	/api/justificatifs Soumettre un justificatif Corps : idRapport, commentaire, urlDoc Réponse : { ID } <i>Note : Parent soumet. idDirecteur null jusqu'à validation.</i>	
GET	/api/disciplines Types de discipline Réponse : { disciplines: [{ ID, libelle, points }] }	

LIB	Bibliothèque	7 endpoints
Table(s) BD : Livres + Specialite		
GET	/api/livres Liste des livres Corps : ?search, ?idSpecialite Réponse : { livres: [{ idLivre, titre, auteurs, prix, libelleSpecialite, totalCopie, edition, annee_parution }] } <i>Note : JOIN Specialite.</i>	
GET	/api/livres/:idLivre Un livre précis Réponse : { livre }	

POST	/api/livres Ajouter un livre Corps : titre, auteurs, prix, idSpecialite, edition, annee_parution, totalCopie Réponse : { idLivres } <i>Note : idAdmin extrait du JWT.</i>
PUT	/api/livres/:idLivres Modifier un livre Corps : titre, auteurs, prix, totalCopie Réponse : { message }
DELETE	/api/livres/:idLivres Supprimer un livre Réponse : { message }
GET	/api/specialites Liste des spécialités Réponse : { specialites: [{ idSpecialite, libelle }] }
POST	/api/specialites Créer une spécialité Corps : libelle Réponse : { idSpecialite }

STATS	Statistiques & Rapports Admin Table(s) BD : Aggregation multi-tables 2 endpoints
GET	/api/admin/statistiques ★ Critique Stats globales du dashboard admin Réponse : { stats: { totalEleves, totalEnseignants, totalClasses, paiementsMois, totalMessages, tauxPaiement } } <i>Note : Aggrégation de plusieurs tables.</i>
GET	/api/admin/rapports/statistiques Stats pour la page Rapports Réponse : { stats: { tauxPresence, moyenneGenerale, tauxPaiement } }

REF	Données de Référence Table(s) BD : VilleNaissance + Quartier + Residents + Titulaire + Fréquentation 18 endpoints
GET	/api/villes-naissance Villes de naissance Corps : ?actif Réponse : { villes: [{ idVille, libelle }] }
POST	/api/villes-naissance Ajouter une ville Corps : libelle Réponse : { idVille }
GET	/api/quartiers Liste des quartiers Réponse : { quartiers: [{ idQuartier, libelle, description }] }

POST	/api/quartiers Créer un quartier Corps : libelle, description Réponse : { idQuartier }
GET	/api/residents Résidents (personnes liées à un quartier) Corps : ?idQuartier Réponse : { residents: [...] } <i>Note : JOIN Personne + Quartier.</i>
POST	/api/residents Lier une personne à un quartier Corps : idPers, idQuartier, description Réponse : { idResi }
DELETE	/api/residents/:idResi Supprimer un résident Réponse : { message }
GET	/api/titulaires Enseignants titulaires d'une salle Corps : ?idSalle Réponse : { titulaires: [...] } <i>Note : JOIN Personne + Salle.</i>
POST	/api/titulaires Nommer un titulaire Corps : idPers, idSalle Réponse : { idTitulaire }
PUT	/api/titulaires/:idTitulaire Modifier un titulaire Corps : idSalle, actif Réponse : { message }
DELETE	/api/titulaires/:idTitulaire Supprimer un titulaire Réponse : { message }
GET	/api/frequentations Fréquentations (inscriptions salle/année) Corps : ?matricule, ?idAca Réponse : { frequentations: [...] }
POST	/api/frequentations Inscrire un élève dans une salle Corps : idSalle, idAcademi, matricule, commentaire Réponse : { idFrequente }
PUT	/api/frequentations/:idFrequente Modifier une fréquentation Corps : commentaire Réponse : { message }

GET**/api/personnes****Liste des personnes**

Corps : ?typePersonne

Réponse : { personnes: [...] }

*Note : Table Personne. typePersonne: 1=Ens, 2=Admin, 3=Scolarite, 4=Parent.***POST****/api/personnes****Créer une personne**

Corps : nom, prenom, dateNaissance, mobile, typePersonne, username, password

Réponse : { idPers }

PUT**/api/personnes/:idPers****Modifier une personne**

Corps : champs

Réponse : { message }

DELETE**/api/personnes/:idPers****Supprimer une personne**

Réponse : { message }

Règles importantes pour l'implémentation

Authentification JWT	Tous les endpoints sauf /api/auth/login doivent vérifier le JWT dans le header "Authorization: Bearer ". Extraire idPers, typePersonne et typeAdmin pour les contrôles d'accès.
Double table d'auth	Le login vérifie d'abord la table Admin (ID, username, password, typeAdmin), puis si non trouvé la table Personne (idPers, username, password, typePersonne). Le token retourné doit inclure le type pour que le frontend redirige correctement.
Format de réponse uniforme	Toujours retourner { success: true, data } en cas de succès et { success: false, message: "..." } en cas d'erreur. Les codes HTTP: 200 (OK), 201 (créé), 400 (mauvaise requête), 401 (non authentifié), 403 (interdit), 404 (non trouvé), 500 (erreur serveur).
Gestion des rôles	typeAdmin: 0=root, 1=Admin, 2=Fondateur, 3=Directeur. typePersonne: 1=Enseignant, 2=Administratif, 3=Scolarité, 4=Parent. Seuls les admins peuvent valider paiements, messages, salaires. Un enseignant ne peut modifier que ses propres notes (idPers).
Endpoints ★ Critiques	21 endpoints sont marqués critiques car ils impliquent des jointures complexes (ex: mes-enfants joint Parents + Eleve + Frequenté) ou des règles métier (ex: initier un paiement notifie l'admin, le parent ne valide pas lui-même).
CORS	Autoriser les requêtes depuis http://localhost:5173 (frontend Vite) en développement. En production, restreindre au domaine du frontend.
Hachage des mots de passe	Les mots de passe ne doivent JAMAIS être stockés en clair. Utiliser bcrypt (cost=10). Le champ password dans Admin et Personne contient le hash.
Pagination	Les endpoints qui retournent des listes (élèves, paiements, rapports) doivent supporter ?page et ?limit. Retourner aussi { total, page, totalPages }.